AI / ML Learning Roadmap

Beginner -> RAG / LLM · Free · Project-Based · ~10-12 months
Built around your 50-question AIML exam syllabus

Contents — click to jump

Context

You're a Java developer preparing via a 50-question AIML exam (linear algebra, calculus, probability, classic ML, neural nets, CNN/RNN/LSTM, autoencoders/VAE/GAN, transformers, RAG/LLM). You want a **fully free**, **project-first** path that (a) onboards you to Python + the scientific stack, (b) builds **solid math foundations** with visual/illustrated guides (StatQuest, 3Blue1Brown), and (c) produces **resume-worthy projects** across LLM/RAG/agents, classic ML/data-science, NLP, and computer vision — ending on RAG/LLM. You also asked for **inline links**, **free "build-your-own" project exercises** (since CodeCrafters is paid), and **industry-recognized certifications** to do in parallel. Project domains are anchored in your **cybersecurity** career shift and your **finance / personal-investing** interest (see *Domain Tracks*), and the **LLM track goes in-depth** — internals -> fine-tuning -> quantization -> serving -> security — not just usage (see *Phase 6B*).

Pace assumed: **~10–15 hrs/week**. Core ML/DL/RAG track ~ 8–9 months; the **full practitioner track** (incl. MLOps + agentic systems + interview prep, Phases 7–8) ~ **10–12 months**. Every phase ships a portfolio artifact. Learning style matched to yours: **illustrated + build-from-scratch**, not hello-world.

> Honesty notes: Josh Starmer's *StatQuest Illustrated Guide* books are **paid**, but the **StatQuest YouTube channel is free** and covers the same material — used throughout. **All core learning below is free.** Certifications (last section) are optional paid exams except the ones explicitly marked **FREE**.

North-Star outcomes (the practitioner you'll become)

This roadmap is built backwards from the capabilities you want, not just the topics:

Outcome you want	How this roadmap builds it
Hear a problem -> judge if AI/ML fits, and how to approach it	"Problem-Framing" thread (below) + a one-page **Decision Lens** written for every project from Phase 3 on
Take it to production (data transform -> training -> deploy, end-to-end)	**Phase 7 — MLOps & Productionization** (pipelines, tracking, API, Docker, CI/CD, monitoring)
Improve a model with confidence (tune/fine-tune existing or build new)	"Model-Improvement" thread (error analysis, bias/variance, data-centric) + the **Build-Your-Own** from-scratch series, so you're **not tool-dependent**
Design & get the best from agents/tools (high demand)	**Phase 6** (RAG/agents) + **Phase 8 — Agentic Systems** (architectures, multi-agent, eval, production)
Clear the toughest interviews with practical, usable knowledge	**Phase 8 — Interview Readiness** (ML system design, ML/DL concepts, DSA, portfolio narrative, mock interviews)

Dual competency principle: the from-scratch challenges make you *independent* (you can build it yourself); Phases 6–8 make you *leveraged* (you get the best out of agents/tools). You'll be able to do both — and, crucially, to **decide which is appropriate**.

Recurring practitioner threads (run alongside the phases, not as separate time)

A) Problem-Framing & ML Judgment — *start Phase 3, apply forever.* For every project, write a one-page **Decision Lens**: Is ML even needed (vs rules/heuristics)? What's the simplest baseline? Is the data sufficient/labeled? What metric maps to business value? What's the cost of being wrong?

- Google — **Rules of Machine Learning** (43 best practices) — https://developers.google.com/machine-learning/guides/rules-of-ml
- Google — **ML Problem Framing** guide — https://developers.google.com/machine-learning/problem-framing

- Andrew Ng — **Machine Learning Yearning** (free PDF) — https://info.deeplearning.ai/machine-learning-yearning-book

B) Model Improvement & Independence — *start Phase 3, deepen through Phase 7.* The discipline of making a model better: **error analysis** (look at the misses), bias/variance diagnosis, train/dev/test strategy, regularization, hyperparameter tuning, transfer learning, and the **decide-to-fine-tune-vs- build-new** judgment. Your Build-Your-Own series guarantees you can implement the internals yourself, so improvement is never a black box you depend on a tool for.

- Andrew Ng — **Machine Learning Yearning** (the canonical "how to improve a model" book; error analysis, ceilings, bias/variance) — https://info.deeplearning.ai/machine-learning-yearning-book
- Chip Huyen — **MLOps guide** (free, practical) — https://huyenchip.com/mlops/

C) Responsible AI & Interpretability — *apply from Phase 3 on.* Fairness/bias auditing, **interpretability** (SHAP, LIME, permutation importance), privacy (differential privacy, federated learning), and model cards. Especially important in **security & finance**, where decisions must be explainable and auditable.

- Christoph Molnar — **Interpretable Machine Learning** (free book) — https://christophm.github.io/interpretable-ml-book/
- Google — **ML Crash Course: Fairness** module — https://developers.google.com/machine-learning/crash-course/fairness
- Amazon ML University — **Responsible AI** (free) — https://aws.amazon.com/machine-learning/mlu/

How this roadmap maps to your exam syllabus

Exam topics (Q#)	Where it's covered
Python: pandas/numpy/mutability/min-max (Q2–5)	Phase 0–1
Linear algebra: inner product, orthogonality, eigenvalues (Q1, Q10)	Phase 2
Calculus: local max, critical points, derivatives (Q6, Q15)	Phase 2
Optimization: gradient/steepest descent, Adagrad/RMSprop (Q7, Q8, Q35)	Phase 2 + Phase 4
Probability/Bayes: Bayes theorem, discriminant functions (Q12, Q13)	Phase 2 + Phase 3
Error metrics: MAE, RSS, F1, cross-validation (Q9, Q14, Q22, Q23)	Phase 3
Classic ML: trees, boosting/bagging, k-means, PCA, sup/unsup (Q11, Q16–Q31)	Phase 3
Neural nets: MLP, backprop, activations, epoch, XOR (Q26–Q39)	Phase 4
CNN/RNN/LSTM/GRU (Q42–Q45)	Phase 4
Autoencoders / VAE / GAN (Q41, Q48–Q50)	Phase 4 + Phase 6
Transformers: positional encoding, residual connections (Q46, Q47)	Phase 5
RAG / LLM	Phase 6

The 50 exam questions are a **sample, not the whole field** — they skip entire areas (reinforcement learning, diffusion/generative vision, time-series, statistics & experimentation, responsible AI, advanced CV/NLP). The map below covers the **whole landscape**; items tagged *(beyond the 50 Qs)* are deliberately added so you learn the field, not just the exam.

Complete Topic Coverage Map (the whole field, not just the exam)

Mathematics & Statistics

- Linear algebra: vectors, matrices, eigen-decomposition, SVD, orthogonality — Phase 2
- Calculus, gradients, optimization (convexity, Lagrange multipliers) — Phase 2
- Probability, distributions, MLE/MAP, Bayes — Phase 2–3
- *(beyond)* Statistics & experimentation: hypothesis testing, confidence intervals, **A/B testing**, bootstrapping — Phase 2–3
- *(beyond)* Information theory: entropy, **cross-entropy, KL divergence**, mutual information — Phase 2

Classic Machine Learning

- Linear/logistic regression, regularization (L1/L2/elastic-net) — Phase 3
- *(beyond)* **SVM & the kernel trick**, Naive Bayes, KNN — Phase 3
- Decision trees, bagging/Random Forest, boosting (**XGBoost/LightGBM/CatBoost**), stacking — Phase 3
- Clustering: k-means, *(beyond)* **hierarchical, DBSCAN, GMM/EM** — Phase 3
- Dimensionality reduction: PCA, *(beyond)* **t-SNE, UMAP, LDA** — Phase 3
- *(beyond)* Imbalanced learning (SMOTE, class weights), **model calibration**, ROC/PR-AUC — Phase 3
- *(beyond)* **Recommender systems** (collaborative/content, matrix factorization) — Phase 3 project
- *(beyond)* **Time-series forecasting** (ARIMA, ETS, Prophet, seasonality) — Phase 3–4 (key for finance)
- Anomaly detection — Phase 4 + cyber track

Deep Learning

- MLP, backprop, activations, optimizers — Phase 4
- *(beyond)* Regularization: **dropout, batch/layer norm**, weight decay, early stopping, data augmentation — Phase 4
- *(beyond)* Initialization (Xavier/He), LR schedules/warmup, gradient clipping — Phase 4
- CNNs; *(beyond)* **advanced CV: ResNet, object detection (YOLO/R-CNN), segmentation (U-Net)** — Phase 4 (CS231n)
- RNN/LSTM/GRU, seq2seq, attention — Phase 4–5
- Autoencoders, VAE, GAN — Phase 4/6
- *(beyond)* **Self-supervised & contrastive learning**, representation/embeddings — Phase 4–5
- *(beyond)* **Graph Neural Networks** — Optional Module C

NLP

- *(beyond)* Classical NLP: tokenization, **TF-IDF, n-grams, word2vec/GloVe**, NER, topic modeling — Phase 5
- Transformers, attention, positional encoding — Phase 5
- *(beyond)* Eval metrics: **BLEU, ROUGE, perplexity** — Phase 5

Generative AI & LLMs

- Transformer internals, tokenization, KV cache, decoding — Phase 5–6B
- Fine-tuning (LoRA/QLoRA/PEFT), alignment (RLHF/DPO/ORPO) — Phase 6B
- Quantization, serving (vLLM/llama.cpp), evaluation — Phase 6B
- RAG, embeddings, vector search; agents & multi-agent systems — Phase 6/8
- *(beyond)* **Diffusion models & image generation** (Stable Diffusion); **multimodal (CLIP)** — Optional Module B
- LLM security (OWASP LLM Top 10) — Phase 6B

Reinforcement Learning *(beyond — absent from the 50 Qs)*

- MDPs, value/policy iteration, **Q-learning, DQN, policy gradients, actor-critic, PPO** — Optional Module A

Systems, MLOps & Responsible AI

- Data pipelines, experiment tracking, API, Docker, CI/CD, monitoring/drift — Phase 7
- *(beyond)* **Responsible AI**: fairness/bias, **interpretability (SHAP/LIME)**, privacy (differential privacy, federated learning), model cards — Thread C

Detailed per-topic study guides

The phases below are the **overview**. For each phase there's a companion guide in [`phases/`](phases/README.md) that breaks the phase into **specific topics**, maps **each topic to the exact video/chapter/article** that covers it (so a course's own flow can't derail you), and adds ■ **checkpoint exercises** to verify understanding before moving on. Built so far: Phase 0, 1, 2 (more being added). Use the overview here to see the arc; use the phase guides to actually study.

Phase 0 — Python for a Java dev + DSA bridge (~2–3 weeks)

Goal: Become fluent in Python idioms (you know the CS, you need the language): dynamic typing, slicing, comprehensions, mutability (your exam Q5!), unpacking, modules, virtualenv, and **OOP-in-Python vs Java** (dunder methods, properties, duck typing).

Free resources

- Official Python Tutorial — https://docs.python.org/3/tutorial/
- Real Python (free articles) — https://realpython.com (search "Python for Java developers", "OOP")
- Corey Schafer — Python channel (OOP playlist is the best on classes) — https://www.youtube.com/@coreyms
- *Automate the Boring Stuff* (free online book) — https://automatetheboringstuff.com
- freeCodeCamp — Python full course — https://www.youtube.com/@freecodecamp (search "Python full course")
- DSA in Python (you know DSA from Java): NeetCode — https://neetcode.io · free book *Problem Solving with Algorithms and Data Structures using Python* — https://runestone.academy/ns/books/published/pythonds/index.html

Project (not hello-world): `finlytics` — a CLI personal-finance analyzer. Ingests CSV bank/credit statements -> categorizes -> monthly summaries -> exports charts. Exercises OOP, file I/O, dataclasses, argparse, pytest. Ship on GitHub with README + tests. **Build-your-own challenge: BYO-1 (autograd engine)** — see challenge series below.

Checkpoint: You write idiomatic Python classes + a packaged CLI without Java habits.

Phase 1 — Scientific Python stack + charts/graphs (~3–4 weeks)

Goal: NumPy (vectorization, broadcasting, `axis` — exam Q4), Pandas (Series/DataFrame — Q2, cleaning, groupby, joins), Matplotlib + Seaborn (you asked about plotting), Jupyter/Colab/Kaggle notebooks (your **free GPU** later).

Free resources

- **Kaggle Learn** (hands-on micro-courses, free + completion certs) — https://www.kaggle.com/learn -> *Python, Pandas, Data Visualization*
- *Python Data Science Handbook*, Jake VanderPlas (**free, full book**) — https://jakevdp.github.io/PythonDataScienceHandbook/

- Corey Schafer — Pandas & Matplotlib playlists — https://www.youtube.com/@coreyms
- freeCodeCamp — NumPy / Pandas courses — https://www.youtube.com/@freecodecamp

Package focus: `numpy`, `pandas`, `matplotlib`, `seaborn`, Jupyter, Google Colab (https://colab.research.google.com), Kaggle Notebooks.

Project (Data Science piece #1): Real-world EDA + data pipeline. Pick a messy public/scraped dataset -> clean -> a polished **EDA notebook** with 8–10 well-labeled visualizations + written narrative. This is the notebook recruiters skim.

Checkpoint: Raw CSV -> clean, well-visualized analysis end-to-end.

Phase 2 — Math foundations (visual-first) (~5–6 weeks, partly parallel with Phase 1)

Goal: Intuition for exam Q1, Q6, Q7, Q8, Q10, Q12, Q13, Q15, Q35.

Free resources (illustrated — your style)

- **3Blue1Brown** — *Essence of Linear Algebra* & *Essence of Calculus* — https://www.youtube.com/@3blue1brown
- **StatQuest with Josh Starmer** — *Statistics Fundamentals* + probability — https://www.youtube.com/@statquest
- **Khan Academy** — Linear Algebra, Multivariable Calculus, Statistics & Probability — https://www.khanacademy.org
- *Mathematics for Machine Learning*, Deisenroth et al. (**free PDF**) — https://mml-book.github.io

Cover: vectors/matrices, dot/inner products & orthogonality, eigenvalues/eigenvectors, SVD, derivatives/gradients, gradient descent, probability, Bayes, distributions, expectation/variance.

Also covers (beyond the 50 Qs): statistics & experimentation — **hypothesis testing, confidence intervals, A/B testing, bootstrapping**; and information theory — **entropy, cross-entropy, KL divergence, mutual information** (the basis for most ML loss functions).

Projects (from-scratch math notebooks):

- Gradient-descent visualizer (NumPy + Matplotlib animation).
- Linear regression from scratch (normal equation *and* gradient descent).
- PCA from scratch via eigen-decomposition (exam Q10 + Q18).

Build-your-own challenges: BYO-2 (regression engine), **BYO-4 (K-Means + PCA)**.

Checkpoint: You can explain and code a gradient, an eigenvector, and a Bayes update.

Phase 3 — Classic Machine Learning (~6–8 weeks)

Goal: Heart of your exam (Q9, Q11, Q14, Q16–Q31): supervised vs unsupervised, trees, bagging/RF, boosting, k-means, PCA, cross-validation, metrics, regularization, bias–variance.

Also covers (beyond the 50 Qs): SVM & the kernel trick, Naive Bayes, KNN; hierarchical/DBSCAN/GMM clustering; t-SNE/UMAP/LDA; imbalanced learning (SMOTE) & calibration; recommender systems; and **time-series forecasting** (ARIMA/ETS/Prophet — *Forecasting: Principles & Practice*, https://otexts.com/fpp3/ — important for your finance track).

Free resources

- **StatQuest — Machine Learning playlist** — https://www.youtube.com/@statquest (your favorite; covers most exam ML topics)
- **Andrew Ng — Machine Learning Specialization** (DeepLearning.AI on Coursera, **audit free**) — https://www.coursera.org/specializations/machine-learning-introduction

- **Google — Machine Learning Crash Course** (free) — https://developers.google.com/machine-learning/crash-course
- ***An Introduction to Statistical Learning (ISLP)*** — **free PDF + free videos** — https://www.statlearning.com
- **Kaggle Learn** — *Intro to ML, Intermediate ML, Feature Engineering* — https://www.kaggle.com/learn
- **scikit-learn** user guide — https://scikit-learn.org/stable/user_guide.html

Package focus: `scikit-learn`, `pandas`, `shap` (explainability).

Projects (Data Science centerpieces): 1. **End-to-end ML pipeline — churn / loan-default prediction.** EDA -> feature engineering -> **cross-validation** -> compare LogReg/RF/Gradient Boosting -> tune -> **SHAP** -> deploy as a **Streamlit app** (free: https://streamlit.io/cloud or Hugging Face Spaces). 2. **Kaggle competition entry** (Titanic -> a live tabular comp) — https://www.kaggle.com/competitions 3. **Recommendation system** (collaborative filtering) with ranking metrics. **Build-your-own challenge: BYO-3 (Decision Tree + Random Forest)**.

Checkpoint: Frame a problem, pick the right model + metric, validate honestly, explain predictions.

Phase 4 — Deep Learning foundations (build-from-scratch) (~6–8 weeks)

Goal: Exam Q26–Q45: MLP, backprop, activations, epochs, XOR, optimizers (SGD/Adagrad/RMSprop/Adam), CNNs, RNN/LSTM/GRU, autoencoders.

Also covers (beyond the 50 Qs): regularization (dropout, batch/layer norm, weight decay, early stopping, data augmentation); initialization, LR schedules/warmup, gradient clipping; transfer learning; **advanced CV** (ResNet, object detection YOLO/R-CNN, segmentation U-Net — via Stanford CS231n, https://cs231n.github.io); self-supervised/contrastive learning & embeddings.

Free resources

- **Andrej Karpathy — *Neural Networks: Zero to Hero*** (build NN -> GPT from scratch; *do in full*) — https://karpathy.ai/zero-to-hero.html · code https://github.com/karpathy/nn-zero-to-hero
- **StatQuest — Neural Networks / Deep Learning** — https://www.youtube.com/@statquest
- **3Blue1Brown — Neural Networks** series — https://www.youtube.com/@3blue1brown
- **fast.ai — Practical Deep Learning for Coders** (free) — https://course.fast.ai
- ***Dive into Deep Learning (d2l.ai)*** — free interactive PyTorch book — https://d2l.ai
- **PyTorch** official tutorials — https://pytorch.org/tutorials

Package focus: `pytorch`, `torchvision`; free GPU via Colab / Kaggle.

Projects (Computer Vision + from-scratch): 1. **Neural net from scratch in NumPy** (micrograd-style) — forward/backprop, train on real data. 2. **CNN image classifier** + transfer learning on a meaningful dataset (plant-disease, X-ray, custom scrape) -> deploy with **Gradio on Hugging Face Spaces** (free) — https://huggingface.co/spaces 3. **Autoencoder** for denoising / anomaly detection (sets up VAE/GAN). **Build-your-own challenges: BYO-5 (mini-PyTorch framework)**, **BYO-6 (CNN from scratch)**.

Checkpoint: Build, train, debug, and deploy a PyTorch model on free GPU.

Phase 5 — NLP + Transformers (~4–6 weeks)

Goal: Exam Q46–Q47 (positional encoding, residual connections, attention) + practical NLP.

Also covers (beyond the 50 Qs): classical NLP foundations — tokenization, **TF-IDF, n-grams, word2vec/GloVe** (gensim), NER, POS tagging, topic modeling (LDA); seq2seq & beam search; and generation

metrics (**BLEU, ROUGE, perplexity**).

Free resources

- **Hugging Face — NLP Course** (free, excellent) — https://huggingface.co/learn/nlp-course
- **Jay Alammar — *The Illustrated Transformer*** (illustrated — your style) — https://jalammar.github.io/illustrated-transformer/
- **Karpathy — "Let's build GPT"** (in Zero-to-Hero) — build a transformer from scratch
- *(optional)* Stanford **CS224n** lectures (free, YouTube) — https://www.youtube.com/@stanfordonline

Package focus: `transformers`, `datasets`, `tokenizers`, `sentence-transformers` (Hugging Face).

Projects (NLP): 1. **Fine-tune a transformer** for a real task (toxic-comment, news-topic, or a **resume parser/classifier**) -> push to **HF Hub**. 2. **Summarizer or NER app** on HF Spaces (Gradio). **Build-your-own challenges: BYO-7 (BPE tokenizer)**, **BYO-8 (mini-GPT)**.

Checkpoint: Fine-tune + deploy a transformer; explain attention end-to-end.

Phase 6 — LLMs, RAG & Agents (capstone) (~6–8 weeks)

Goal: Your end-goal: embeddings, vector search, retrieval, prompt engineering, tool-calling agents, evaluation, light fine-tuning (LoRA).

Free resources

- **Hugging Face — LLM Course + Agents Course** — https://huggingface.co/learn/llm-course · https://huggingface.co/learn/agents-course
- **DeepLearning.AI short courses (FREE):** *LangChain for LLM App Dev*, *LangChain: Chat with Your Data*, *Building & Evaluating Advanced RAG*, *Functions, Tools & Agents with LangChain* — https://www.deeplearning.ai/short-courses/
- **Google + Kaggle — 5-Day Gen AI Intensive** (FREE, self-paced) — https://www.kaggle.com/learn-guide/5-day-genai
- **freeCodeCamp — "RAG from Scratch"** (LangChain engineer) — https://www.youtube.com/@freecodecamp
- **LangChain Academy** (free, official) — https://academy.langchain.com
- **Activeloop — free RAG course** — https://learn.activeloop.ai/courses/rag
- **Ollama** (run open LLMs locally, free) — https://ollama.com

Package focus: `langchain`/`langgraph`, `llama-index`, `chromadb`/`faiss`, `sentence-transformers`, `ollama`, free API tiers (this Claude Code env also exposes the **Claude API**).

Projects (resume centerpieces): 1. **"Chat with your documents" RAG app** — PDFs -> chunk -> embed -> **Chroma/FAISS** -> retrieve -> answer with **citations** -> Streamlit/Gradio UI on HF Spaces. *Flagship piece.* 2. **Tool-using AI agent** (LangGraph) — research/coding assistant with tool-calling + memory. 3. **LoRA fine-tune** a small open LLM on free Colab/Kaggle GPU. 4. **RAG evaluation harness** (RAGAS — https://docs.ragas.io) — faithfulness/relevance metrics. **Build-your-own challenges: BYO-9 (RAG engine from scratch)**, **BYO-10 (ReAct agent)**, **BYO-11 (vector DB)**.

Checkpoint: A deployed, **evaluated** RAG system + a working agent — every component explainable.

Phase 6B — LLM In-Depth (internals -> fine-tune -> quantize -> serve -> secure) (~5–7 weeks)

Goal: Go beyond *using* LLMs to *understanding and shaping* them. This is what separates an "API caller" from an "LLM engineer" in interviews. Builds directly on **BYO-7 (tokenizer)** and **BYO-8 (mini-GPT)** — you'll have already built a transformer, so now you go production-deep.

Free resources

- **Maxime Labonne — LLM Course** (Fundamentals / Scientist / Engineer; roadmaps + Colab notebooks) — https://github.com/mlabonne/llm-course
- **Sebastian Raschka — Build an LLM From Scratch** (free code repo; book is paid) — https://github.com/rasbt/LLMs-from-scratch
- **Stanford CS336 — Language Modeling from Scratch** (2025 lectures, free) — https://stanford-cs336.github.io
- **Andrej Karpathy — "Deep Dive into LLMs like ChatGPT"** + "Let's build the GPT Tokenizer" (free) — https://www.youtube.com/@AndrejKarpathy
- **Hugging Face — LLM Course** — https://huggingface.co/learn/llm-course
- **LLM Visualization** (watch a transformer run, interactive) — https://bbycroft.net/llm
- **3Blue1Brown — "But what is a GPT?" / "Attention in transformers"** — https://www.youtube.com/@3blue1brown

Topics & tooling

- *Internals:* tokenization (BPE), embeddings, attention + **KV cache**, positional encodings, decoding (temperature, top-k/p), context windows, scaling laws, mixture-of-experts.
- *Adaptation:* SFT, **LoRA / QLoRA / PEFT**, instruction tuning, alignment (**RLHF, DPO, ORPO**). Tools: HF `trl`, `peft`, `unsloth` (2–5x faster QLoRA on free Colab), `axolotl`.
- *Efficiency:* **quantization** (GGUF via **llama.cpp**; GPTQ/AWQ 4-bit), `bitsandbytes`; serving with **vLLM** / TGI; local models via Ollama / llama.cpp.
- *Evaluation:* **lm-evaluation-harness**, `promptfoo`, **LLM-as-judge**, RAG eval (RAGAS).
- *Security (ties to your cyber pivot):* **OWASP LLM Top 10** — prompt injection, insecure output handling, excessive agency, etc. + basic red-teaming — https://genai.owasp.org/llm-top-10/

Projects: 1. **QLoRA fine-tune** an open LLM (Llama/Mistral/Qwen) on a domain dataset (security Q&A *or* finance/earnings) on free Colab -> push to HF Hub -> evaluate before/after. 2. **Quantize + serve** it (GGUF via llama.cpp / Ollama) and benchmark latency/throughput vs full precision. 3. **LLM evaluation harness** — compare 2–3 models on a task with lm-eval-harness + an LLM-as-judge rubric. **Build-your-own challenges: BYO-7, BYO-8** (already), plus optional **BYO-16 (LLM red-team probe)**.

Checkpoint: You can explain a transformer end-to-end, fine-tune + quantize + serve an open model, evaluate it, and reason about its security.

Phase 7 — MLOps & Productionization (data -> training -> deploy -> monitor) (~5–6 weeks)

Goal: The "reach production" skill — turn a notebook model into a deployed, monitored service. Data pipelines, **experiment tracking**, model packaging, **REST APIs**, **Docker**, **CI/CD**, **monitoring & drift detection**, batch vs real-time serving. This is what separates "did a Kaggle notebook" from "shipped ML."

Free resources

- **Made With ML** (design -> develop -> deploy -> iterate; the best free production-ML course) — https://madewithml.com
- **MLOps Zoomcamp** (DataTalks.Club; MLflow, Docker, deployment, monitoring; free, self-paced) — https://github.com/DataTalksClub/mlops-zoomcamp
- **Full Stack Deep Learning** (free) — https://fullstackdeeplearning.com/course/
- **Chip Huyen — MLOps guide** — https://huyenchip.com/mlops/ *(her book "Designing ML Systems" is excellent but paid)*
- Google — **MLOps** docs / Rules of ML — https://developers.google.com/machine-learning/guides/rules-of-ml
- Tools: **FastAPI** (https://fastapi.tiangolo.com), **MLflow** (https://mlflow.org) or **Weights & Biases** free tier (https://wandb.ai), **Docker**, GitHub Actions.

Project (productionization capstone): Take your **Phase 3 churn model** (or CV model) and ship it properly: data prep pipeline -> **MLflow experiment tracking** -> packaged model -> **FastAPI** inference service -> **Dockerized** -> **CI/CD** (GitHub Actions: test + build) -> deployed (HF Spaces / free cloud) -> **monitoring + drift detection** (Evidently AI, free). Document the full lifecycle in the README — this is a standout resume item that most candidates lack.

Checkpoint: You can take any model from notebook -> versioned, tested, deployed, monitored service.

Phase 8 — Agentic Systems (design + production) + Interview Readiness (~5–7 weeks)

Goal: Master the high-demand skill of **designing agentic systems** (not just calling an agent library) and convert everything into **interview-ready** capability.

Part A — Agentic systems (design-level)

- **Anthropic — Building Effective AI Agents** (workflows vs agents, when NOT to use an agent, patterns: prompt-chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer) — https://www.anthropic.com/research/building-effective-agents · cookbook patterns: https://github.com/anthropics/anthropic-cookbook/tree/main/patterns/agents
- **Hugging Face — Agents Course** — https://huggingface.co/learn/agents-course
- **LangGraph** docs + **LangChain Academy** (stateful, multi-agent orchestration) — https://academy.langchain.com
- **DeepLearning.AI (free) agent short courses:** *AI Agents in LangGraph*, *Multi AI Agent Systems with crewAI, Long-Term Agentic Memory with LangGraph* — https://www.deeplearning.ai/short-courses/
- Cover: tool design, memory (short/long-term), planning, multi-agent orchestration, **agent evaluation & guardrails**, **cost/latency tradeoffs, and the judgment of** workflow vs agent**.

Project (agentic centerpiece): A **multi-agent system** with LangGraph — e.g., a research assistant (planner -> researcher(s) -> writer -> critic) with tools, long-term memory, retries, and an **evaluation harness**. Pair it with **BYO-10/BYO-13** (you build the ReAct loop *and* a multi-agent orchestrator from scratch) so you can speak to internals *and* frameworks in interviews.

Part B — Interview readiness

- **Chip Huyen — Introduction to Machine Learning Interviews** (free book; process, concepts, ML system design) — https://huyenchip.com/ml-interviews-book/
- **ML system design** practice (frame -> data -> model -> eval -> serving -> monitoring) — reuse the Decision Lens habit from the Problem-Framing thread.
- **DSA** refresh for coding rounds — https://neetcode.io
- **Portfolio narrative:** for each project, prepare a 2-minute "problem -> approach -> result -> what I'd improve" story. Do **2–3 mock interviews** (peers / Pramp-style / record yourself).

Checkpoint: You can design an agentic system on a whiteboard, justify workflow-vs-agent choices, and narrate every portfolio project under interview pressure.

Optional specialization modules (add by interest / target role)

These cover major areas the 50 questions skip. Slot them in where they fit (RL & diffusion sit well after Phase 4–6); do the ones that match your target roles.

Module A — Reinforcement Learning (~4–5 weeks)

Why: absent from your exam, but core to robotics, game AI, **RLHF**, and agentic decision-making.

- **Hugging Face — Deep RL Course** (hands-on, free, certificate) — https://huggingface.co/learn/deep-rl-course
- **OpenAI — Spinning Up in Deep RL** — https://spinningup.openai.com
- **Sutton & Barto — Reinforcement Learning: An Introduction** (free PDF) — http://incompleteideas.net/book/the-book.html
- **David Silver — RL Course** (DeepMind, YouTube) · practice in **Gymnasium** — https://gymnasium.farama.org
- Cover: MDPs, value/policy iteration, Q-learning, **DQN**, policy gradients, actor-critic, **PPO**.
- **Project:** train a DQN/PPO agent on a Gym environment; for your domains, a **trading-env** agent or a security decision game.

Module B — Diffusion & Generative Vision (~3–4 weeks)

Why: image/audio/video generation is a huge slice of modern GenAI; complements your LLM depth.

- **Hugging Face — Diffusion Models Course** (free) — https://huggingface.co/learn/diffusion-course
- **fast.ai — Part 2** (build Stable Diffusion from scratch) — https://course.fast.ai/Lessons/part2.html
- **Jay Alammar — The Illustrated Stable Diffusion** — https://jalammar.github.io/illustrated-stable-diffusion/
- Cover: forward/reverse diffusion, U-Net, guidance, latent diffusion, **multimodal (CLIP)**.
- **Project:** fine-tune Stable Diffusion (DreamBooth/LoRA) on a small concept; ship a Gradio demo.

Module C — Graph Neural Networks (~3 weeks)

Why: fraud rings, recommendation, knowledge graphs, and **network security** (graphs of hosts/flows).

- **Stanford CS224W — Machine Learning with Graphs** (free lectures) — https://web.stanford.edu/class/cs224w/
- **PyTorch Geometric** — https://pytorch-geometric.readthedocs.io
- **Project:** node classification / **fraud-ring detection** on a graph — a strong cyber/finance crossover.

> Advanced computer vision (ResNet, object detection, segmentation) is best learned via **Stanford CS231n** (free) — https://cs231n.github.io — fold it into Phase 4 if CV is a target.

Domain Tracks — Cybersecurity & Finance (use these as your projects)

Because you've moved into **cybersecurity** and want **finance / personal-investing** skills, do the per-phase projects in *these* domains so you learn ML **and** the domain together. Pick the cyber or finance variant (or both) at each phase; the two flagship capstones are resume-defining and the finance one doubles as a real tool for your own investing.

> **Not financial advice.** Finance projects here are for **analysis, research, and backtesting only** — never wire them to execute live trades or move money. Backtests overstate real results; always use walk-forward validation and account for fees/slippage.

Cybersecurity datasets & resources (free)

- **CIC-IDS2017 / CSE-CIC-IDS2018, NSL-KDD, UNSW-NB15** (network intrusion) — UNB CIC: https://www.unb.ca/cic/datasets/
- **PhishTank** URLs, **EMBER** (malware features), Kaggle security datasets.
- **OWASP GenAI / LLM Top 10** — https://genai.owasp.org/llm-top-10/ · **MITRE ATT&CK** — https://attack.mitre.org

Finance datasets & tools (free)

- Market data: `yfinance` — https://github.com/ranaroussi/yfinance · **FRED** — https://fred.stlouisfed.org · Stooq.
- Backtesting: **backtesting.py** (simple), **Backtrader** (realistic) — https://www.backtrader.com, **vectorbt** (fast) — https://github.com/polakowo/vectorbt.

- Portfolio/analytics: **PyPortfolioOpt** — https://github.com/robertmartin8/PyPortfolioOpt · **quantstats / pyfolio** (tear sheets).
- Filings: **SEC EDGAR** (10-K/10-Q) — https://www.sec.gov/edgar.

Per-phase mapping (Cybersecurity | Finance)

Phase	Cybersecurity project	Finance project
1 EDA	Network-traffic / log EDA (CIC-IDS)	Stock & portfolio EDA via yfinance (returns, volatility, correlation)
3 Classic ML	Intrusion / phishing / malware classifier (CV + SHAP)	Credit-default or fraud detection; stock-direction baseline (leakage-aware)
4 Deep Learning	Deep **anomaly detection** on logs/traffic (autoencoder)	LSTM/temporal model for volatility/returns (strict walk-forward)
5 NLP	Phishing-email / malicious-intent classifier; log parsing	Financial-news & earnings-call **sentiment**; NER on filings
6 RAG/Agents	Security assistant — RAG over **CVE / MITRE ATT&CK** + SOC triage agent	Investing research assistant — RAG over **10-K / earnings** with citations
6B LLM depth	QLoRA fine-tune on security Q&A; **OWASP red-team**	QLoRA fine-tune on financial Q&A; structured extraction from filings
7 MLOps	Productionize the intrusion detector (monitored API + drift)	Productionize a scoring/forecasting service (monitored)
8 Agentic	Multi-agent **SOC / incident-response** crew (planner->analyst->reporter)	Multi-agent **investment-research** crew (screener->analyst->risk->writer)

Flagship capstones

- **Cyber:** an LLM-assisted **security analyst** — RAG over CVE/ATT&CK + a triage agent that classifies alerts, explains them, and drafts a report — *plus* an **OWASP LLM Top 10 mini red-team** of your own RAG app (prompt injection, output handling, excessive agency).
- **Finance:** a **personal investing analysis dashboard** — pull your holdings (yfinance), compute risk/return (**Sharpe, max drawdown**), run **Monte Carlo** projections of expected outcomes, **Markowitz** efficient-frontier optimization, and a **backtested** strategy with an honest tear sheet — *plus* a RAG assistant over filings to support decisions. Quantify expected return, risk, and the distribution of outcomes **before** you invest, instead of guessing.

"Build Your Own AI" challenge series (free, CodeCrafters-style — I'll author these)

CodeCrafters is paid and not AI-focused. Free general alternatives exist — **Coding Challenges by John Crickett** (https://codingchallenges.fyi), the **build-your-own-x** list (https://github.com/codecrafters-io/build-your-own-x) — but none cover ML/DL/RAG. So I'll create a **staged, test-driven, from-scratch series** (each stage adds functionality + has acceptance tests, just like CodeCrafters). On approval I scaffold each as a GitHub repo with a starter, a stage checklist, and `pytest` tests. Core logic is hand-built (no high-level libs) to prove understanding.

ID	Build Your Own...	Stages (high level)	Reinforces	Phase
BYO-1	**Autograd engine** (micrograd-style)	`Value` -> ops(+,*,tanh) -> topological backprop -> MLP -> train	Backprop Q27/Q34	0–1
BYO-2	**Regression engine**	hypothesis -> MSE/cross-entropy -> gradient descent -> L1/L2 -> mini-batch SGD	RSS/MAE Q9/Q14/Q35	2

ID	Build Your Own...	Stages (high level)	Reinforces	Phase
BYO-3	**Decision Tree + Random Forest**	Gini/entropy -> best split -> recursive tree -> predict -> bagging -> forest	Trees/boosting Q11/Q30	3
BYO-4	**K-Means + PCA**	distances -> assign -> update -> k-means++ ; covariance -> eigen -> project	Q10/Q18/Q31	2–3
BYO-5	**Mini-PyTorch framework**	tensor+autograd -> Linear/ReLU/Softmax -> loss -> SGD/Adam -> train MNIST	Q26–Q39	4
BYO-6	**CNN from scratch**	conv op -> pooling -> forward -> conv-backprop -> train small images	Q44/Q45	4
BYO-7	**BPE Tokenizer** (minBPE-style)	vocab -> pair counts -> merges -> encode/decode	LLM prep	5
BYO-8	**Mini-GPT** (char transformer)	token+positional embed -> self-attention -> multi-head -> block(residual+LN) -> train	Q46/Q47	5
BYO-9	**RAG engine from scratch** (no LangChain)	loader/chunker -> embeddings -> cosine vector search -> retriever -> prompt -> LLM -> citations -> eval	RAG	6
BYO-10	**ReAct AI agent**	tool registry -> tool-aware prompt -> parse call -> execute -> observe loop -> memory	Agents	6
BYO-11	**Vector database** (optional)	store -> brute-force search -> ANN index (IVF/HNSW-lite) -> persistence	Systems depth	6
BYO-12	**Autoencoder -> VAE -> tiny GAN** (optional)	encoder/decoder -> reconstruction -> VAE reparam -> GAN gen/disc loop	Q41/Q48–Q50	4–6
BYO-13	**Multi-agent orchestrator** (from scratch)	agent base -> message bus -> roles (planner/worker/critic) -> shared memory -> eval loop	Agentic design	8
BYO-14	**Intrusion / anomaly detector** (from scratch)	features -> distance/isolation score -> threshold -> precision/recall/ROC	Cyber, metrics Q23	3–4
BYO-15	**Backtesting engine** (from scratch)	price feed -> signal -> positions -> PnL/returns -> Sharpe/drawdown	Finance	3
BYO-16	**LLM red-team probe** (optional)	prompt-injection corpus -> attack runner -> detector -> OWASP report	LLM security	6B

Reference implementations to study (not copy): Karpathy **micrograd** (https://github.com/karpathy/micrograd), **minBPE** (https://github.com/karpathy/minbpe), **nanoGPT** (https://github.com/karpathy/nanoGPT).

Certifications to do in parallel (industry-recognized, in-demand)

Do these **alongside** the phases — they validate skills for recruiters. Free ones first.

FREE certificates (do during the matching phase):

- **freeCodeCamp — Machine Learning with Python** (free cert; TF/CNN/RNN/NLP projects) — https://www.freecodecamp.org/learn/machine-learning-with-python/ -> Phase 3–4
- **Kaggle Learn** completion certificates (Python, Pandas, ML, DL, Feature Eng.) — https://www.kaggle.com/learn -> Phases 1–4
- **Hugging Face** course certificates (NLP / LLM / Agents) — https://huggingface.co/learn -> Phases 5–6
- **Google — ML Crash Course** badge — https://developers.google.com/machine-learning/crash-course -> Phase 3
- **Google + Kaggle 5-Day Gen AI** completion — https://www.kaggle.com/learn-guide/5-day-genai -> Phase 6

PAID but high-ROI / widely accepted (optional; take after the relevant phase):

- **AWS Certified AI Practitioner (AIF-C01)** — beginner-friendly, strong market signal — https://aws.amazon.com/certification/certified-ai-practitioner/ -> after Phase 3
- **Google Cloud — Professional Machine Learning Engineer** — top recognition for ML roles — https://cloud.google.com/learn/certification/machine-learning-engineer -> after Phase 4–5
- **AWS Certified Machine Learning Engineer – Associate** (replaced the old ML Specialty) — https://aws.amazon.com/certification/certified-machine-learning-engineer-associate/ -> after Phase 4
- **Microsoft Azure AI Engineer** — note **AI-102 retires 30 Jun 2026**; take the successor **AI-103 (Azure AI App & Agent Developer Associate)** which adds agents/GenAI — https://learn.microsoft.com/credentials/ -> after Phase 6

> Skip the **TensorFlow Developer Certificate** — Google **discontinued** it (May 2024). The ecosystem is PyTorch-first, which this roadmap already uses. Course completion "certificates" from Coursera audits are not the same as proctored cloud certs — prioritize the cloud creds above for resume weight once your projects exist.

Recommended sequence: free certs as you go (Kaggle, freeCodeCamp, HF) -> one **beginner cloud cert** (AWS AI Practitioner) mid-way -> one **flagship cloud cert** (Google PMLE *or* AWS ML Engineer Associate) near the end, once you have deployed projects to back it up.

Your cyber edge: combining ML with your **cybersecurity** background is a rare, in-demand profile. **AI/LLM security** (OWASP LLM Top 10, prompt-injection red-teaming, securing agentic systems) is emerging fast and few people can do both — lean into it. If you also hold/pursue core security certs (e.g. Security+, or cloud security creds), pair them with the AI certs above to stand out for **AI-security / ML-security engineer** roles.

Cross-cutting: tools, free compute & showcase (ongoing)

- **Git/GitHub** from Phase 0; every project = its own repo with a strong README (problem, approach, results, screenshots, live link).
- **Free compute:** Google Colab (free GPU), Kaggle Notebooks (free GPU + datasets).
- **Free deployment:** Streamlit Community Cloud, **Hugging Face Spaces** (Gradio/Streamlit), GitHub Pages.
- **Write as you learn** (matches your illustrated style): a short blog post per project — portfolio + reinforcement.

Portfolio target by the end

GitHub + HF Spaces with **~8 polished deployed projects** (built in your **cyber/finance** domains) + **the BYO challenge repos** showing from-scratch internals: clean EDA, classic-ML app w/ explainability, from-scratch neural net, deployed anomaly detector, fine-tuned NLP model, a **RAG app**, a **fine-tuned + quantized + served LLM**, a **productionized service** (API+Docker+CI+monitoring), and a **multi-agent system** — with the two **flagship capstones** (security analyst + personal investing dashboard) as centerpieces. Plus a **Decision Lens one-pager** per project, free certs (Kaggle/freeCodeCamp/HF), and ideally one cloud cert. This portfolio *is* your interview material.

Milestones / how to verify progress

1. **Phase 1:** raw CSV -> clean, visualized EDA notebook on GitHub. 2. **Phase 2:** gradient descent + linear regression + PCA from scratch (BYO-2, BYO-4). 3. **Phase 3:** deployed ML app (CV + SHAP) + a Kaggle entry + first **Decision Lens** one-pager; freeCodeCamp/Kaggle certs. 4. **Phase 4:** NumPy NN + deployed CNN on HF Spaces (BYO-5, BYO-6); first **error-analysis** writeup. 5. **Phase 5:** fine-tuned transformer on HF Hub + deployed NLP app (BYO-7, BYO-8); HF cert. 6. **Phase 6:** deployed **evaluated** RAG app + agent (BYO-9, BYO-10); Google 5-Day GenAI. 6B. **Phase 6B:** an open LLM **fine-tuned (QLoRA) + quantized + served** + an eval comparison; can explain a transformer end-to-end. 7. **Phase 7:** one model fully **productionized** (MLflow + FastAPI + Docker + CI/CD + monitoring); MLOps Zoomcamp / Made With ML done. 8. **Phase 8: multi-agent system** (BYO-13) + ML **system-design** writeups + 2–3 **mock interviews**; can argue workflow-vs-agent. 9. **Domain capstones:** the **security analyst** (RAG+agent+red-team) and the **personal investing dashboard** (risk/return + Monte Carlo + backtest) both shipped. 10. **Cross-check:** re-take the 50-question exam — reason through every item, not memorize.

What I'll produce on approval

1. A polished **clickable PDF** of this roadmap (index -> North-Star outcomes -> phase pages -> clickable resource links -> challenge series -> certifications -> milestone checklist), in the style of your exam PDF. 2. **Scaffold the BYO challenge repos** (starter code + stage checklist + `pytest` tests) — starting with BYO-1 (autograd) so you can begin immediately. 3. A reusable **Decision Lens template** (the problem-framing one-pager) for every future project. 4. *(Optional)* Save a copy of the roadmap to your Obsidian vault.

Appendix — Build Status (challenges & capstones)

All 16 **Build-Your-Own** challenges are scaffolded and **test-verified** (starter stubs + staged `pytest` tests, each validated against a reference solution that was then removed). Implement the stubs until the tests pass. Located in `challenges/`.

#	Challenge	Tests	Status
BYO-1	autograd engine (backprop)	19	Done
BYO-2	regression (linear/logistic, GD, L1/L2)	6	Done
BYO-3	decision tree + random forest	5	Done
BYO-4	k-means + PCA	4	Done
BYO-5	mini-PyTorch (autograd framework + MLP)	6	Done
BYO-6	CNN (conv forward + backprop)	5	Done
BYO-7	BPE tokenizer	6	Done

#	Challenge	Tests	Status
BYO-8	mini-GPT (attention, LayerNorm, residual)	7	Done
BYO-9	RAG engine (no LangChain)	6	Done
BYO-10	ReAct tool-using agent	5	Done
BYO-11	vector database (brute-force + ANN)	4	Done
BYO-12	autoencoder -> VAE -> GAN	7	Done
BYO-13	multi-agent orchestrator	4	Done
BYO-14	intrusion / anomaly detector (cyber)	4	Done
BYO-15	backtesting engine (finance)	8	Done
BYO-16	LLM red-team probe (OWASP LLM01)	5	Done

Capstones (`capstones/`):

Capstone	Focus	Tests	Status
DQN trading agent	Reinforcement Learning x Finance	7	Done
GNN fraud-ring detector	Graph Neural Networks x Cyber/Finance	6	Done

Total: **16 challenges (101 tests) + 2 capstones (13 tests)**, all reference-verified. Build progress is also tracked in `challenges/README.md`.